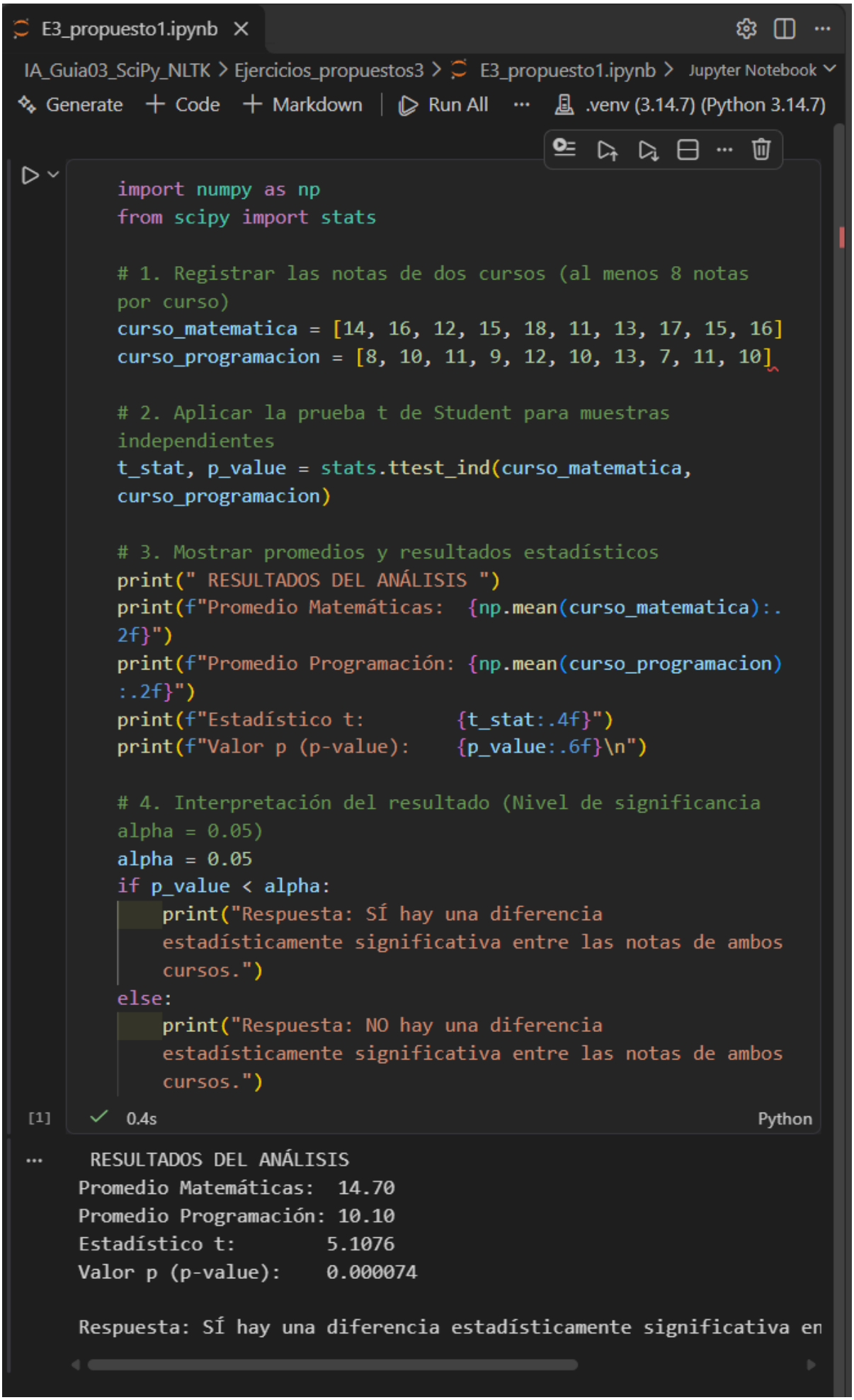
CLASE 3: TAREA 3

1. SciPy: registra las notas de dos de tus cursos (al menos 8 notas por curso) y aplica ttest_ind. ¿Hay diferencia significativa entre ambos cursos?



2. SciPy: usa optimize.minimize_scalar para hallar el mínimo de f(x) = x^2 – 6x + 13 y comprueba el resultado derivando a mano.

E3_propuesto2.ipynb

Guia03_SciPy_NLTK > Ejercicios_propuestos3 > E3_propuesto2.ipynb > in Jupyter Notebook

Generate + Code + Markdown | Run All ... .venv (3.14.7) (Python 3.14.7)

```python
import numpy as np
from scipy import optimize

# 1. Definir la función f(x) = x² - 6x + 13
def f(x):
    return x**2 - 6*x + 13

# 2. Encontrar el mínimo utilizando minimize_scalar
resultado = optimize.minimize_scalar(f)

# 3. Mostrar los resultados de SciPy
x_min = resultado.x
y_min = resultado.fun

print(" RESULTADO CON SCIPY ")
print(f"Mínimo encontrado en x = {x_min:.4f}")
print(f"Valor mínimo de f(x)  = {y_min:.4f}\n")

# 4. Comprobación matemática a mano
print(" COMPROBACIÓN MANUAL (DERIVADA) ")
print("1. Función original:   f(x) = x² - 6x + 13")
print("2. Primera derivada:   f'(x) = 2x - 6")
print("3. Igualar a cero:     2x - 6 = 0  =>  2x = 6  =>  x = 3")
print("4. Segunda derivada:   f''(x) = 2  (al ser > 0, se confirma que es un MÍNIMO)")
print(f"5. Evaluar en f(3):    f(3) = (3)² - 6(3) + 13 = 9 - 18 + 13 = {f(3)}")
```

[1] ✓ 0.2s Python

```
 RESULTADO CON SCIPY
Mínimo encontrado en x = 3.0000
Valor mínimo de f(x)  = 4.0000

 COMPROBACIÓN MANUAL (DERIVADA)
1. Función original:   f(x) = x² - 6x + 13
2. Primera derivada:   f'(x) = 2x - 6
3. Igualar a cero:     2x - 6 = 0  =>  2x = 6  =>  x = 3
4. Segunda derivada:   f''(x) = 2  (al ser > 0, se confirma que es
5. Evaluar en f(3):    f(3) = (3)² - 6(3) + 13 = 9 - 18 + 13 = 4
```

3. SciPy: resuelve con linalg.solve el sistema: x + y + z = 6, 2x – y + z = 3, x + 2y – z = 2.

E3_propuesto3.ipynb

Ejercicios_propuestos3 > E3_propuesto3.ipynb > import numpy as np Jupyter Notebook

Generate + Code + Markdown | Run All ... .venv (3.14.7) (Python 3.14.7)

```python
import numpy as np
from scipy import linalg
# 1. Definir la matriz de coeficientes (A)
A = np.array([
    [1,  1,  1],
    [2, -1,  1],
    [1,  2, -1]])
# 2. Definir el vector de términos independientes (b)
b = np.array([6, 3, 2])
# 3. Resolver el sistema A * x = b
solucion = linalg.solve(A, b)
# 4. Extraer los valores de x, y, z
x, y, z = solucion
# 5. Mostrar los resultados
print("SOLUCIÓN DEL SISTEMA DE ECUACIONES ")
print(f"x = {x:.2f}")
print(f"y = {y:.2f}")
print(f"z = {z:.2f}\n")
# 6. Comprobación de las ecuaciones
print(" COMPROBACIÓN ")
print(f"Ecuación 1 (x + y + z = 6):      {x} + {y} + {z} =
{x + y + z}")
print(f"Ecuación 2 (2x - y + z = 3):     2({x}) - ({y}) +
({z}) = {2*x - y + z}")
print(f"Ecuación 3 (x + 2y - z = 2):     {x} + 2({y}) -
({z}) = {x + 2*y - z}")
```

[1] ✓ 0.1s Python

```
SOLUCIÓN DEL SISTEMA DE ECUACIONES
x = 1.00
y = 2.00
z = 3.00

 COMPROBACIÓN
Ecuación 1 (x + y + z = 6):      1.0 + 2.0 + 3.0 = 6.0
Ecuación 2 (2x - y + z = 3):     2(1.0) - (2.0) + (3.0) = 3.0
Ecuación 3 (x + 2y - z = 2):     1.0 + 2(2.0) - (3.0) = 2.0
```

4. NLTK: copia un párrafo de una noticia en español, obtén las 10 palabras más frecuentes (sin stopwords) y muéstralas en un gráfico con frecuencia.plot(10).

E3_propuesto4.ipynb

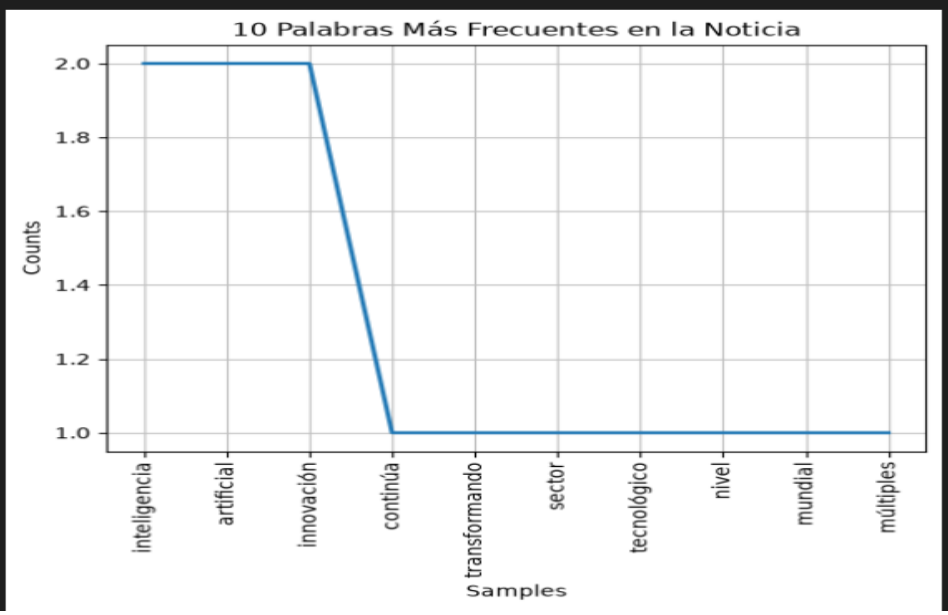
IA_Guia03_SciPy_NLTK > Ejercicios_propuestos3 > E3_propuesto4.ipynb > Jupyter Notebook

Generate + Code + Markdown | Run All ... .venv (3.14.7) (Python 3.14.7)

```python
import nltk
import matplotlib.pyplot as plt
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.probability import FreqDist
# Asegurar la descarga de recursos necesarios de NLTK
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
# 1. Párrafo de una noticia en español
noticia = (
    "La inteligencia artificial continúa transformando el
    sector tecnológico a nivel mundial. "
    "Múltiples empresas de tecnología e innovación están
    invirtiendo recursos significativos "
    "para desarrollar nuevos modelos de inteligencia
    artificial que optimicen procesos, "
    "mejoren la productividad e impulsen la innovación
    digital en diversos sectores de la industria.")
# 2. Tokenizar y limpiar (convertir a minúsculas y filtrar
solo palabras)
tokens = word_tokenize(noticia.lower(), language="spanish")
palabras = [t for t in tokens if t.isalpha()]
# 3. Eliminar stopwords (palabras vacías)
vacias = set(stopwords.words("spanish"))
palabras_limpias = [p for p in palabras if p not in vacias]
# 4. Obtener las 10 palabras más frecuentes
frecuencia = FreqDist(palabras_limpias)
print(" 10 PALABRAS MÁS FRECUENTES ")
for palabra, cant in frecuencia.most_common(10):
    print(f"{palabra}: {cant}")
# 5. Mostrar el gráfico con frecuencia.plot(10)
frecuencia.plot(10, cumulative=False, title="10 Palabras
Más Frecuentes en la Noticia")
plt.show()
```

[1] ✓ 8.2s Python

```
 10 PALABRAS MÁS FRECUENTES
inteligencia: 2
artificial: 2
innovación: 2
continúa: 1
transformando: 1
sector: 1
tecnológico: 1
nivel: 1
mundial: 1
múltiples: 1
```



5. NLTK: agrega 10 opiniones más al clasificador del Paso 13 (5 positivas y 5 negativas) y aplica stemming en extraer_caracteristicas. ¿Mejora la clasificación de “La clase fue aburrida”?

E3_propuesto5.ipynb

IA_Guia03_SciPy_NLTK > Ejercicios_propuestos3 > E3_propuesto5.ipynb > import nltk

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | .venv (3.14.7) (Python 3.14.7)

```python
import nltk
from nltk.stem import SnowballStemmer
from nltk.tokenize import word_tokenize
from nltk.classify import NaiveBayesClassifier
from nltk.classify.util import accuracy

# 1. Definir el conjunto de datos ampliado (Opiniones originales + 10 nuevas)
datos_entrenamiento = [
    # Opiniones iniciales (Paso 13)
    ("Excelente clase, aprendí mucho", "positivo"),
    ("El profesor explica muy bien y claro", "positivo"),
    ("Buena explicación y gran contenido", "positivo"),
    ("Pésima clase, no me gustó para nada", "negativo"),
    ("Muy confuso y nada claro el tema", "negativo"),
    ("No entendí nada de la explicación", "negativo"),

    # 5 Nuevas opiniones positivas
    ("La clase estuvo fantástica e interesante", "positivo"),
    ("Gran metodología y excelentes ejercicios", "positivo"),
    ("Disfruté mucho la lección de hoy", "positivo"),
    ("El tema es fascinante y muy bien explicado", "positivo"),
    ("Me encantó el curso, sumamente útil", "positivo"),

    # 5 Nuevas opiniones negativas
    ("La clase fue aburrida y monótona", "negativo"),
    ("Perdí el tiempo, estuvo horrible", "negativo"),
    ("No me agradó la presentación del tema", "negativo"),
    ("Poca claridad y contenido deficiente", "negativo"),
    ("La sesión fue muy aburrida y pesada", "negativo")
]

# 2. Inicializar el stemmer en español
stemmer = SnowballStemmer("spanish")

# 3. Función de extracción de características APLICANDO STEMMING
def extraer_caracteristicas(texto):
    tokens = word_tokenize(texto.lower(), language="spanish")
    # Convertir cada palabra a su raíz (stemming)
    raices = [stemmer.stem(palabra) for palabra in tokens if palabra.isalpha()]
    return {raiz: True for raiz in raices}

# 4. Procesar el conjunto de datos de entrenamiento
datos_procesados = [
    (extraer_caracteristicas(texto), etiqueta)
    for (texto, etiqueta) in datos_entrenamiento
]

# 5. Entrenar el clasificador Naive Bayes
clasificador = NaiveBayesClassifier.train(datos_procesados)

# 6. Probar con la frase objetivo "La clase fue aburrida"
frase_prueba = "La clase fue aburrida"
caracteristicas_prueba = extraer_caracteristicas(frase_prueba)
resultado = clasificador.classify(caracteristicas_prueba)

# 7. Mostrar resultados
print("EVALUACIÓN DEL CLASIFICADOR")
print(f"Frase a evaluar: '{frase_prueba}'")
print(f"Características (Raíces extraídas): {list(caracteristicas_prueba.keys())}")
print(f"Resultado de la clasificación: {resultado.upper()}\n")

# Mostrar las características más informativas
print("CARACTERÍSTICAS MÁS INFORMATIVAS")
clasificador.show_most_informative_features(5)
```

[1] ✓ 0.8s Python

```
EVALUACIÓN DEL CLASIFICADOR
Frase a evaluar: 'La clase fue aburrida'
Características (Raíces extraídas): ['la', 'clas', 'fue', 'aburr']
Resultado de la clasificación: NEGATIVO

CARACTERÍSTICAS MÁS INFORMATIVAS
Most Informative Features
                  explic = True           positi : negati =      2.3 : 1.0
                      la = True           negati : positi =      1.8 : 1.0
                      me = True           negati : positi =      1.7 : 1.0
                     tem = True           negati : positi =      1.7 : 1.0
                     nad = None           positi : negati =      1.5 : 1.0
```

6. Reto integrador: agrega dos documentos más al Paso 14 y, para una pregunta escrita por el usuario con input(), muestra cuál documento es el más parecido.

